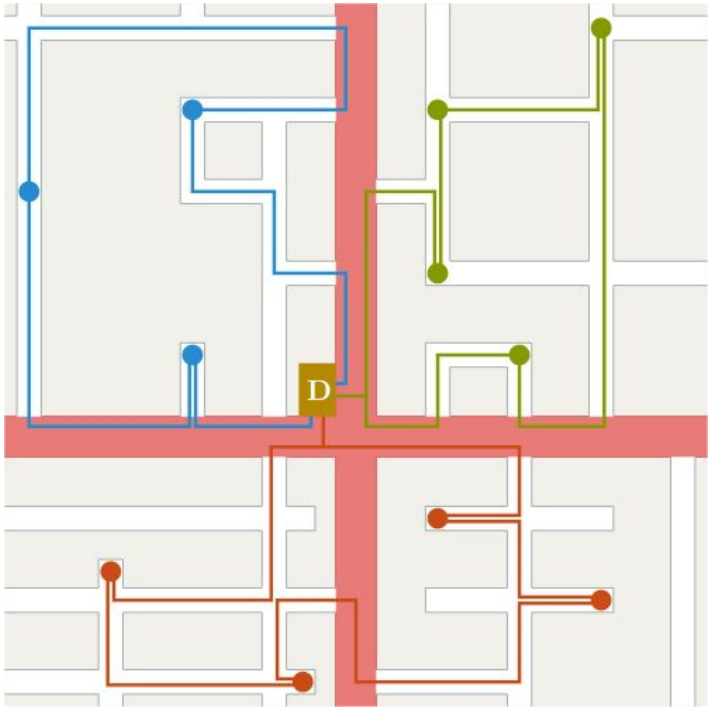
Delivery Routing

Vehicle Routing Problem

Find an optimal set of routes for trucks to traverse in order to deliver demands to customers

- minimize the total route cost (e.g., total travel distance)

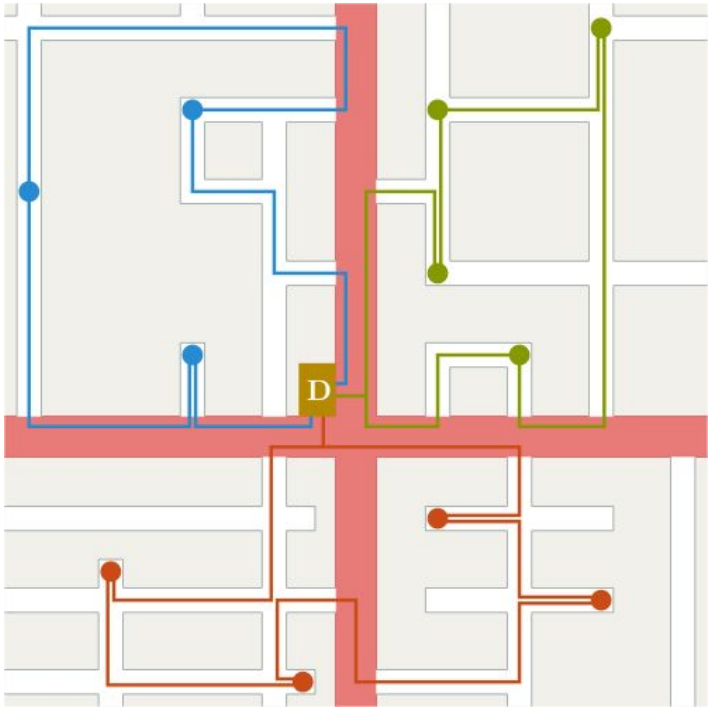


https://en.wikipedia.org/wiki/Vehicle_routing_problem

Capacitated Vehicle Routing Problem

Find an optimal set of routes for trucks to traverse in order to deliver demands to customers

- minimize the total route cost (e.g., total travel distance)
- **each truck has a limited capacity**



https://en.wikipedia.org/wiki/Vehicle_routing_problem

Problem Instance

Find an optimal set of routes for **two trucks** to traverse in order to deliver demands to **20 locations**

- minimize the total travel distance
- each truck has a limited capacity of **100**

- **a distance matrix of 21x21**
- **a demand list of 20**

Optimization Constraints

Find an optimal set of routes for **two trucks** to traverse in order to deliver demands to **20 locations**

- minimize the total travel distance
- each truck has a limited capacity of **100**

- **visit each location exactly once**
- **respect each truck's capacity limit**
- **assign each truck no more than one route**

Computational Complexity

CVRP is hard by a reduction from the TSP

- could be formulated as a binary integer programming model
- a metaheuristic-based solver generally performs well

Google OR-Tools

Google OR-Tools

OR-Tools OR API

Search /

Eng

Installation Guides Reference Examples Support

Route. Schedule. Plan. Assign. Pack. **Solve.**

OR-Tools is fast and portable software for combinatorial optimization.

Get started with OR-Tools

Learn how to solve optimization problems from C++, Python, C#, or Java.

Get started

Install OR-Tools

See the Release Notes for the latest updates.

Install OR-Tools

OR-Tools won gold in the international constraint programming competition every year since 2013.

About OR-Tools

OR-Tools is an open source software suite for optimization, tuned for tackling the world's toughest problems in vehicle routing, flows, integer and linear programming, and constraint programming.

After modeling your problem in the programming language of your choice, you can use any of a half dozen solvers to solve it: commercial solvers such as CPLEX, Gurobi or Fico Xpress, or open-source solvers such as SCIP, GLPK or Google's GLOP and award-winning CP-SAT.

Google AI

Capacitated Vehicle Routing Problem

Google OR-Tools | OR-Tools | OR API | Search / | Engl

OR-Tools

Installation | Guides | Reference | Examples | Support

Filter

- Network Flows
- Linear Optimization
- Integer Optimization
- Assignment
- Packing
- Routing
  - Overview
  - Traveling Salesperson Problem
  - Vehicle Routing Problem
  - Capacity Constraints
  - Pickups and Deliveries
  - Time Window Constraints
  - Dimensions
  - Resource Constraints
  - Penalties and Dropping Visits
  - Common Routing Tasks
  - Routing Options
  - Google Directions API
  - Original CP Solver

Home > Products > OR-Tools > Guides

Was this helpful?

Capacity Constraints

Send feedback

Page Summary

The *capacitated vehicle routing problem* (CVRP) is a VRP in which vehicles with limited carrying capacity need to pick up or deliver items at various locations. The items have a quantity, such as weight or volume, and the vehicles have a maximum **capacity** that they can carry. The problem is to pick up or deliver the items for the least cost, while never exceeding the capacity of the vehicles.

In the following example, we assume that all items are being picked up. The program that solves this problem also works if all items are being delivered: in this case, you can think of the capacity constraint being applied when the vehicles leave the depot fully loaded. But the capacity constraints are implemented the same way in both cases.

Note: For a related example in which there are both pickups and deliveries, see Pickups and Deliveries.

CVRP example

Next, we describe an example of a VRP with capacity constraints. The example extends the previous VRP example and adds the following requirements. At each location there is a **demand** corresponding to the quantity of the item to be picked up. Also, each vehicle has a maximum capacity of 15. (We aren't specifying units for the demands or capacity.)

The grid below shows the locations to visit in blue and the company location in black. The demands are shown at the lower right of each

On this page

- CVRP example
- Solving the CVRP example with OR-Tools
  - Create the data
  - Add the distance callback
  - Add the demand callback and capacity
  - Add the solution printer
  - Main function
  - Running the program
  - Complete programs
- What happens if a problem has no soluti

Code Summary

```
# Instantiate the data problem.
data = create_data_model()

# Create the routing index manager.
manager = pywrapcp.RoutingIndexManager(
    len(data["distance_matrix"]), data["num_vehicles"], data["depot"]
)

# Create Routing Model.
routing = pywrapcp.RoutingModel(manager)

# Create and register a transit callback.
def distance_callback(from_index, to_index):
    from_node = manager.IndexToNode(from_index)
    to_node = manager.IndexToNode(to_index)
    return data["distance_matrix"][from_node][to_node]

transit_callback_index = routing.RegisterTransitCallback(distance_callback)
```

Code Summary

```
# Define cost of each arc.
routing.SetArcCostEvaluatorOfAllVehicles(transit_callback_index)
```

```
# Add Capacity constraint.
def demand_callback(from_index):
    from_node = manager.IndexToNode(from_index)
    return data["demands"][from_node]
```

```
demand_callback_index = routing.RegisterUnaryTransitCallback(demand_callback)
routing.AddDimensionWithVehicleCapacity(
    demand_callback_index,
    0,
    data["vehicle_capacities"],
    True,
    "Capacity",
)
```

Code Summary

```
# Setting first solution heuristic.
search_parameters = pywrapcp.DefaultRoutingSearchParameters()
search_parameters.first_solution_strategy = (
    routing_enums_pb2.FirstSolutionStrategy.PATH_CHEAPEST_ARC
)
search_parameters.local_search_metaheuristic = (
    routing_enums_pb2.LocalSearchMetaheuristic.GUIDED_LOCAL_SEARCH
)
search_parameters.time_limit.FromSeconds(1)

# Solve the problem.
solution = routing.SolveWithParameters(search_parameters)

# Print solution on console.
if solution:
    print_solution(data, manager, routing, solution)
```

Experimental Results

Objective: 215

Route for truck 0:
0 Load(92) -> 5 Load(83) -> 14 Load(74) -> 6 Load(63) -> 10 Load(53) -> 9 Load(45) -> 19 Load(35) -> 13 Load(30) -> 4 Load(15) -> 3 Load(8) -> 1 Load(0) -> 0
Distance of the route: 106
Load of the route: 92

Route for truck 1:
0 Load(98) -> 11 Load(91) -> 12 Load(78) -> 17 Load(70) -> 8 Load(56) -> 7 Load(50) -> 18 Load(39) -> 16 Load(33) -> 15 Load(21) -> 20 Load(12) -> 2 Load(0) -> 0
Distance of the route: 109
Load of the route: 98

Total distance of all routes: 215
Total load of all routes: 190

Google OR-Tools

Can solve many types of VRPs

- Travelling Salesperson Problem
  a classical routing problem in which there is just only one vehicle
- Vehicle routing problem
  a generalisation of the TSP with multiple vehicles
- VRP with capacity constraints
  vehicles have maximum capacities for the items they can carry
- VRP with time windows
  vehicles must visit the locations in specified time intervals
- VRP with resource constraints
  such as space or personnel to load and unload vehicles at the depot
- VRP with dropped visits
  vehicles aren’t required to visit all locations, but must pay a penalty for each visit that is dropped

How many trucks?

- The lowest number of trucks needed can be computed.
  - total decomands divided by truck capacity

- Minimizing the total route distance also attempts to minimize the number of trucks to be used.
  - Re-run the code with 3 identical trucks as input, which results in no route assigned to the third truck.
  - If a route can be serviced by one truck, it won’t be split into two routes for two truck in the optimal solution.